3.6 The Tuple Relational Calculus

When we write a relational-algebra expression, we provide a sequence of procedures that generates the answer to our query. The tuple relational calculus, by contrast, is a **nonprocedural** query language. It describes the desired information without giving a specific procedure for obtaining that information.

A query in the tuple relational calculus is expressed as

$$\{t \mid P(t)\}$$

that is, it is the set of all tuples t such that predicate P is true for t. Following our earlier notation, we use $t[A]$ to denote the value of tuple t on attribute A, and we use $t \in r$ to denote that tuple t is in relation r.

Before we give a formal definition of the tuple relational calculus, we return to some of the queries for which we wrote relational-algebra expressions in Section 3.2.

3.6.1 Example Queries

Say that we want to find the *branch-name*, *loan-number*, and *amount* for loans of over $1200:

$$\{t \mid t \in loan \wedge t[amount] > 1200\}$$

Suppose that we want only the *loan-number* attribute, rather than all attributes of the *loan* relation. To write this query in the tuple relational calculus, we need to write an expression for a relation on the schema (*loan-number*). We need those tuples on (*loan-number*) such that there is a tuple in *loan* with the *amount* attribute > 1200. To express this request, we need the construct "there exists" from mathematical logic. The notation

$$\exists t \in r\ (Q(t))$$

means "there exists a tuple t in relation r such that predicate $Q(t)$ is true."

Using this notation, we can write the query "Find the loan number for each loan of an amount greater than $1200" as

$$\{t \mid \exists s \in loan\ (t[loan\text{-}number] = s[loan\text{-}number] \wedge s[amount] > 1200)\}$$

In English, we read the preceding expression as "The set of all tuples t such that there exists a tuple s in relation *loan* for which the values of t and s for the *loan-number* attribute are equal, and the value of s for the *amount* attribute is greater than $1200."

Tuple variable t is defined on only the *loan-number* attribute, since that is the only attribute having a condition specified for t. Thus, the result is a relation on (*loan-number*).

Consider the query "Find the names of all customers who have a loan from the Perryridge branch." This query is slightly more complex than the previous queries, since it involves two relations: *borrower* and *loan*. As we shall see, however, all it requires is that we have two "there exists" clauses in our tuple-relational-calculus expression, connected by *and* ($\wedge$). We write the query as follows:

$$\{t \mid \exists\, s \in \mathit{borrower}\ (t[\mathit{customer\text{-}name}] = s[\mathit{customer\text{-}name}] \\ \wedge \exists\, u \in \mathit{loan}\ (u[\mathit{loan\text{-}number}] = s[\mathit{loan\text{-}number}] \\ \wedge u[\mathit{branch\text{-}name}] = \text{“Perryridge”}))\}$$

In English, this expression is “The set of all (*customer-name*) tuples for which the customer has a loan that is at the Perryridge branch.” Tuple variable u ensures that the customer is a borrower at the Perryridge branch. Tuple variable s is restricted to pertain to the same loan number as s. Figure 3.37 shows the result of this query.

To find all customers who have a loan, an account, or both at the bank, we used the union operation in the relational algebra. In the tuple relational calculus, we shall need two “there exists” clauses, connected by *or* ($\vee$):

$$\{t \mid \exists\, s \in \mathit{borrower}\ (t[\mathit{customer\text{-}name}] = s[\mathit{customer\text{-}name}]) \\ \vee \exists\, u \in \mathit{depositor}\ (t[\mathit{customer\text{-}name}] = u[\mathit{customer\text{-}name}])\}$$

This expression gives us the set of all *customer-name* tuples for which at least one of the following holds:

- The *customer-name* appears in some tuple of the *borrower* relation as a borrower from the bank.
- The *customer-name* appears in some tuple of the *depositor* relation as a depositor of the bank.

If some customer has both a loan and an account at the bank, that customer appears only once in the result, because the mathematical definition of a set does not allow duplicate members. The result of this query appeared earlier in Figure 3.12.

If we now want *only* those customers who have *both* an account and a loan at the bank, all we need to do is to change the *or* ($\vee$) to *and* ($\wedge$) in the preceding expression.

$$\{t \mid \exists\, s \in \mathit{borrower}\ (t[\mathit{customer\text{-}name}] = s[\mathit{customer\text{-}name}]) \\ \wedge \exists\, u \in \mathit{depositor}\ (t[\mathit{customer\text{-}name}] = u[\mathit{customer\text{-}name}])\}$$

The result of this query appeared in Figure 3.20.

Now consider the query “Find all customers who have an account at the bank but do not have a loan from the bank.” The tuple-relational-calculus expression for this query is similar to the expressions that we have just seen, except for the use of the *not* ($\neg$) symbol:

$$\{t \mid \exists\, u \in \mathit{depositor}\ (t[\mathit{customer\text{-}name}] = u[\mathit{customer\text{-}name}]) \\ \wedge \neg \exists\, s \in \mathit{borrower}\ (t[\mathit{customer\text{-}name}] = s[\mathit{customer\text{-}name}])\}$$

customer-name
Adams
Hayes

Figure 3.37 Names of all customers who have a loan at the Perryridge branch.

This tuple-relational-calculus expression uses the $\exists\ u\ \in\ depositor\ (\ldots)$ clause to require that the customer have an account at the bank, and it uses the $\neg\ \exists\ s\ \in\ borrower\ (\ldots)$ clause to eliminate those customers who appear in some tuple of the *borrower* relation as having a loan from the bank. The result of this query appeared in Figure 3.13.

The query that we shall consider next uses implication, denoted by $\Rightarrow$. The formula $P\ \Rightarrow\ Q$ means "P implies Q"; that is, "if P is true, then Q must be true." Note that $P\ \Rightarrow\ Q$ is logically equivalent to $\neg P\ \vee\ Q$. The use of implication rather than *not* and *or* often suggests a more intuitive interpretation of a query in English.

Consider the query that we used in Section 3.2.3 to illustrate the division operation: "Find all customers who have an account at all branches located in Brooklyn." To write this query in the tuple relational calculus, we introduce the "for all" construct, denoted by $\forall$. The notation

$$\forall\ t\ \in\ r\ (Q(t))$$

means "Q is true for all tuples t in relation r."

We write the expression for our query as follows:

$$\begin{aligned}\{t \mid \exists\ r\ \in\ &customer\ (r[customer\text{-}name] = t[customer\text{-}name]) \wedge \\ &(\forall\ u\ \in\ branch\ (u[branch\text{-}city]\ =\ \text{“ Brooklyn”}\ \Rightarrow \\ &\quad \exists\ s\ \in\ depositor\ (t[customer\text{-}name]\ =\ s[customer\text{-}name] \\ &\quad \wedge \exists\ w\ \in\ account\ (w[account\text{-}number]\ =\ s[account\text{-}number] \\ &\quad \wedge w[branch\text{-}name]\ =\ u[branch\text{-}name])))\}\end{aligned}$$

In English, we interpret this expression as "The set of all customers (that is, (*customer-name*) tuples *t*) such that, for *all* tuples *u* in the *branch* relation, if the value of *u* on attribute *branch-city* is Brooklyn, then the customer has an account at the branch whose name appears in the *branch-name* attribute of u."

Note that there is a subtlety in the above query: If there is no branch in Brooklyn, all customer names satisfy the condition. The first line of the query expression is critical in this case—without the condition

$$\exists\ r\ \in\ customer\ (r[customer\text{-}name] = t[customer\text{-}name])$$

if there is no branch in Brooklyn, any value of t (including values that are not customer names in the *depositor* relation) would qualify.

3.6.2 Formal Definition

We are now ready for a formal definition. A tuple-relational-calculus expression is of the form

$$\{t \mid P(t)\}$$

where P is a *formula*. Several tuple variables may appear in a formula. A tuple variable is said to be a *free variable* unless it is quantified by a $\exists$ or $\forall$. Thus, in

$$t\ \in\ loan\ \wedge\ \exists\ s\ \in\ customer(t[branch\text{-}name]\ =\ s[branch\text{-}name])$$

t is a free variable. Tuple variable s is said to be a *bound* variable.

A tuple-relational-calculus formula is built up out of *atoms*. An atom has one of the following forms:

- $s \in r$, where s is a tuple variable and r is a relation (we do not allow use of the $\notin$ operator)
- $s[x]\ \Theta\ u[y]$, where s and u are tuple variables, x is an attribute on which s is defined, y is an attribute on which u is defined, and Θ is a comparison operator ($<$, $\leq$, $=$, $\neq$, $>$, $\geq$); we require that attributes x and y have domains whose members can be compared by Θ
- $s[x]\ \Theta\ c$, where s is a tuple variable, x is an attribute on which s is defined, Θ is a comparison operator, and c is a constant in the domain of attribute x

We build up formulae from atoms by using the following rules:

- An atom is a formula.
- If P_1 is a formula, then so are $\neg P_1$ and (P_1).
- If P_1 and P_2 are formulae, then so are $P_1 \vee P_2$, $P_1 \wedge P_2$, and $P_1 \Rightarrow P_2$.
- If $P_1(s)$ is a formula containing a free tuple variable s, and r is a relation, then

  $$\exists\, s \in r\ (P_1(s)) \text{ and } \forall\, s \in r\ (P_1(s))$$

  are also formulae.

As we could for the relational algebra, we can write equivalent expressions that are not identical in appearance. In the tuple relational calculus, these equivalences include the following three rules:

1. $P_1 \wedge P_2$ is equivalent to $\neg\ (\neg(P_1) \vee \neg(P_2))$.
2. $\forall\, t \in r\ (P_1(t))$ is equivalent to $\neg\ \exists\, t \in r\ (\neg P_1(t))$.
3. $P_1 \Rightarrow P_2$ is equivalent to $\neg(P_1) \vee P_2$.

3.6.3 Safety of Expressions

There is one final issue to be addressed. A tuple-relational-calculus expression may generate an infinite relation. Suppose that we write the expression

$$\{t \mid \neg\ (t \in \mathit{loan})\}$$

There are infinitely many tuples that are not in *loan*. Most of these tuples contain values that do not even appear in the database! Clearly, we do not wish to allow such expressions.

To help us define a restriction of the tuple relational calculus, we introduce the concept of the **domain** of a tuple relational formula, P. Intuitively, the domain of P, denoted $dom(P)$, is the set of all values referenced by P. They include values mentioned in P itself, as well as values that appear in a tuple of a relation mentioned in P. Thus, the domain of P is the set of all values that appear explicitly in